

LAB 13 — Contournement de la détection de root Android avec Objection

Introduction

Sur les systèmes Android, certaines applications intègrent des mécanismes de sécurité destinés à vérifier si l'appareil est rooté. Cette vérification vise à limiter les risques liés à un accès privilégié, comme la modification du comportement de l'application, l'accès à des données sensibles ou l'interception de communications.

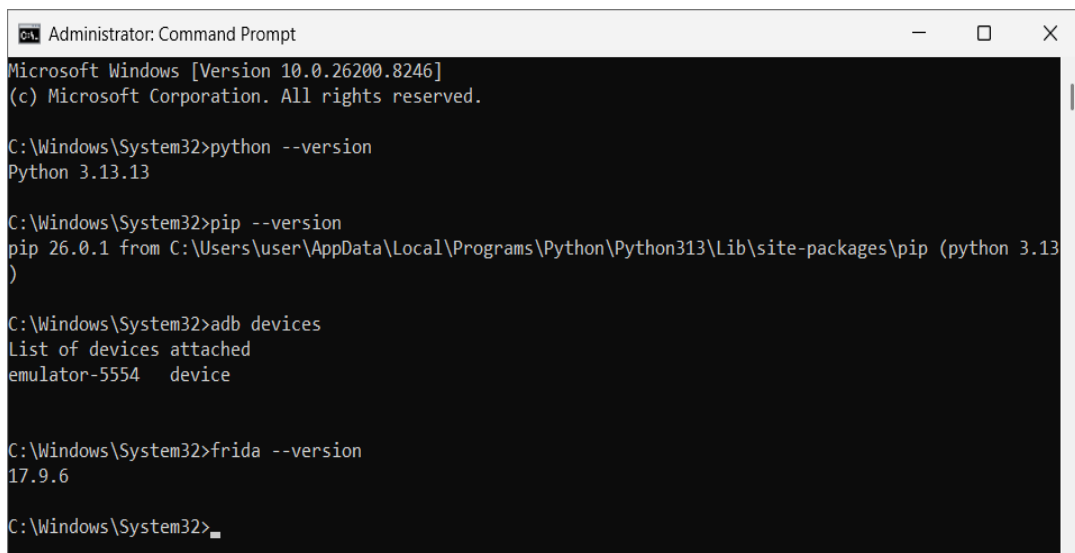
Pour détecter un environnement rooté, les applications s'appuient généralement sur plusieurs techniques : recherche de fichiers spécifiques (comme `su`), exécution de commandes système ou utilisation de bibliothèques spécialisées telles que RootBeer.

Dans un contexte d'audit de sécurité mobile, il est essentiel de comprendre ces mécanismes afin d'évaluer leur efficacité et leur résistance face à des techniques de contournement.

Étape 1 — Installation des outils

L'environnement de travail repose sur deux outils principaux :

- **Frida** : framework d'instrumentation dynamique permettant d'injecter du code dans une application en cours d'exécution



```
Administrator: Command Prompt
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>python --version
Python 3.13.13

C:\Windows\System32>pip --version
pip 26.0.1 from C:\Users\user\AppData\Local\Programs\Python\Python313\Lib\site-packages\pip (python 3.13)

C:\Windows\System32>adb devices
List of devices attached
emulator-5554    device

C:\Windows\System32>frida --version
17.9.6

C:\Windows\System32>
```

- **Objection** : interface en ligne de commande facilitant l'utilisation de Frida

```
C:\Windows\System32>objection --help
Usage: objection [OPTIONS] COMMAND [ARGS]...

  _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _ _
  | . | . | . | . | . | . | . | . | . | . | . | . | . | . | . |
  | _ | _ | _ | _ | _ | _ | _ | _ | _ | _ | _ | _ | _ | _ | _ |
  | _ | (object)inject(ion)

  Runtime Mobile Exploration
  by: @leonjza from @sensepost

Options:
  -N, --network          Connect using a network connection instead of USB.
  -h, --host TEXT        [default: 127.0.0.1]
  -P, --port INTEGER     [default: 27042]
  -ah, --api-host TEXT   [default: 127.0.0.1]
  -ap, --api-port INTEGER [default: 8888]
  -n, --name TEXT        Name or bundle identifier to attach to.
  -S, --serial TEXT       A device serial to connect to.
  -d, --debug            Enable debug mode with verbose output.
  -s, --spawn            Spawn the target.
  -p, --no-pause         Resume the target immediately.
  -f, --foremost         Use the current foremost application.
  --debugger             Enable the Chrome debug port.
  --uid TEXT             Specify the uid to run as (Android only).
  --help                Show this message and exit.

Commands:
  api      Start the objection API server in headless mode.
  patchapk Patch an APK with the frida-gadget.so.
  patchipa Patch an IPA with the FridaGadget.dylib.
  run      Run a single objection command.
  signapk  Zipalign and sign an APK with the objection key.
  start    Start a new session
  version  Prints the current version and exits.

C:\Windows\System32>
```

Ces outils sont installés côté machine (PC) et permettent d'interagir avec l'application cible en temps réel.

Étape 2 — Préparation de l'appareil

Cette étape consiste à :

- Identifier l'architecture du système Android (ABI)
- Télécharger la version correspondante de frida-server
- Transférer le binaire vers l'appareil Android
- Attribuer les permissions nécessaires
- Démarrer le service Frida

Une fois lancé, frida-server permet à la machine hôte de communiquer avec l'application mobile.

```
C:\Windows\System32>adb shell getprop ro.product.cpu.abi
x86_64
```

Étape 3 — Interaction avec l'application cible

L'outil Objection est utilisé pour s'attacher à l'application cible en cours d'exécution.

Deux approches sont possibles :

- **Attachement (attach)** : connexion à une application déjà lancée
- **Démarrage contrôlé (spawn)** : lancement de l'application avec injection préalable

Dans ce laboratoire, Objection permet d'exécuter des commandes directement dans le contexte de l'application.

```
C:\Windows\System32>adb shell ps | findstr frida
root      12354      1  15284020  52764 do_sys_poll      0 S frida-server

C:\Windows\System32>adb shell pkill frida-server

C:\Windows\System32>adb shell "/data/local/tmp/frida-server -l 0.0.0.0:27042"
```

```

C:\Users\user\Downloads\frida-server-17.9.6-android-x86_64>frida-ps -Uai
PID    Name                               Identifier
-----
12223  Chrome                             com.android.chrome
11364  Photos                             com.google.android.apps.photos
1384   SIM Toolkit                         com.android.stk
12664  Settings                           com.android.settings
12375  ezmobile                           com.pwnsec.ezmobile
-     Calendar                           com.google.android.calendar
-     Camera                             com.android.camera2
-     Clock                              com.google.android.deskclock
-     Contacts                           com.google.android.contacts
-     Drive                              com.google.android.apps.docs
-     Files                              com.google.android.documentsui
-     Glasses                           com.google.android.glasses.companion
-     Gmail                              com.google.android.gm
-     Google                             com.google.android.googlequicksearchbox
-     Maps                              com.google.android.apps.maps
-     Messages                           com.google.android.apps.messaging
-     Personal Safety                    com.google.android.apps.safetyhub
-     Phone                             com.google.android.dialer
-     YouTube                           com.google.android.youtube
-     YouTube Music                       com.google.android.apps.youtube.music

C:\Users\user\Downloads\frida-server-17.9.6-android-x86_64>

```

Étape 4 — Fonctionnement de la commande android root disable

La commande android root disable repose sur l'injection de hooks via Frida.

Son objectif est de :

- Intercepter les appels aux fonctions de détection de root
- Modifier dynamiquement leurs résultats
- Masquer les indicateurs de compromission (présence de su, commandes système, etc.)

Ainsi, l'application est amenée à croire qu'elle s'exécute sur un environnement sain, même si le système est réellement rooté.

PID	Name	Identifier
-----	------	------------

PID	Name	Identifier
-----	------	------------

```
C:\Windows\System32>objection -n com.pwnsec.ezmobile start
```

Runtime Mobile Exploration
by: @leonjza from @sensepost

```
com.pwnsec.ezmobile (run) on (Android: 17) [usb] #
```

Étape 5 — Validation du contournement

Après application des hooks, le comportement de l'application est analysé afin de vérifier si :

- Les mécanismes de détection sont neutralisés
- Les restrictions initiales sont levées
- L'application fonctionne normalement sans bloquer l'utilisateur

Cette étape permet de confirmer l'efficacité du bypass.

```
Administrator: Command Prompt - objection -n com.pwnsec.ezmobile start
[tab] for command suggestions
com.pwnsec.ezmobile (run) on (Android: 17) [usb] # android root disable
A Frida agent exception has occurred.
Error: Unable to find copied methods in java/lang/Thread; please file a bug
  at <anonymous> (/src/index.js:3860)
  at <anonymous> (/src/index.js:2847)
  at getArtClassSpec (/src/index.js:3874)
  at compileModule (/src/index.js:9475)
  at ensureInitialized (/src/index.js:9444)
  at build (/src/index.js:9358)
  at _make (/src/index.js:11262)
  at use (/src/index.js:11141)
  at <anonymous> (/src/index.js:13369)
  at <anonymous> (/src/index.js:2847)
  at _performPendingVmOpsWhenReady (/src/index.js:13407)
  at perform (/src/index.js:13348)
  at <anonymous> (/src/index.js:13544)
  at Promise (native)
  at wrapJavaPerform (/src/index.js:13545)
  at testKeysCheck (/src/index.js:20965)
  at disable2 (/src/index.js:21272)
  at androidRootDetectionDisable (/src/index.js:21466)
  at call (native)
  at handleRpcMessage (/frida/runtime/message-dispatcher.js:39)
  at handleMessage (/frida/runtime/message-dispatcher.js:25)

Python stack trace: Traceback (most recent call last):
  File "C:\Users\user\AppData\Local\Programs\Python\Python313\Lib\site-packages\objection\console\repl.py", line 392, in run
    self.run_command(document)
    ~~~~~^
  File "C:\Users\user\AppData\Local\Programs\Python\Python313\Lib\site-packages\objection\console\repl.py", line 170, in run_command
    exec_method(arguments)
    ~~~~~^
  File "C:\Users\user\AppData\Local\Programs\Python\Python313\Lib\site-packages\objection\commands\android\root.py", line 13, in disable
    api.android_root_detection_disable()
    ~~~~~^
  File "C:\Users\user\AppData\Local\Programs\Python\Python313\Lib\site-packages\frida\core.py", line 188, in method
    return script._rpc_request(request, data, **kwargs)
    ~~~~~^
  File "C:\Users\user\AppData\Local\Programs\Python\Python313\Lib\site-packages\frida\core.py", line 94, in wrapper
    return f(*args, **kwargs)
  File "C:\Users\user\AppData\Local\Programs\Python\Python313\Lib\site-packages\frida\core.py", line 499, in _rpc_request
    raise result.error
frida.core.RPCException: Error: Unable to find copied methods in java/lang/Thread; please file a bug
  at <anonymous> (/src/index.js:3860)
  at <anonymous> (/src/index.js:2847)
```

Étape 6 — Automatisation

Afin d'optimiser le processus, il est possible d'automatiser l'injection des commandes dès le lancement de l'application.

Cela permet :

- D'appliquer les hooks automatiquement
- De gagner du temps lors des tests
- D'assurer une exécution cohérente à chaque démarrage

```
projet.fst.ma.starsgallery (run) on (Android: 11) [usb] # android root disable
(agent) Registering job 965661. Name: root-detection-disable
projet.fst.ma.starsgallery (run) on (Android: 11) [usb] # android sslpinnin
g disable
(agent) Custom TrustManager ready, overriding SSLContext.init()
(agent) Found com.android.org.conscrypt.TrustManagerImpl, overriding TrustManagerImpl.verifyChain()
(agent) Found com.android.org.conscrypt.TrustManagerImpl, overriding TrustManagerImpl.checkTrustedRecursive()
(agent) Registering job 919153. Name: android-sslpinning-disable
projet.fst.ma.starsgallery (run) on (Android: 11) [usb] # android hooking s
earch classes root
```

Conclusion

Le contournement de la détection de root repose sur des techniques d'instrumentation dynamique permettant de manipuler le comportement interne des applications.

Grâce à Frida et Objection, il est possible de simuler un environnement sécurisé en interceptant les mécanismes de vérification utilisés par les applications Android.

Toutefois, ces techniques ne suppriment pas réellement le root du système. Elles visent uniquement à tromper l'application dans un cadre de test.